

alx

An Introduction to Data Pipelines



What is a Data Pipeline?



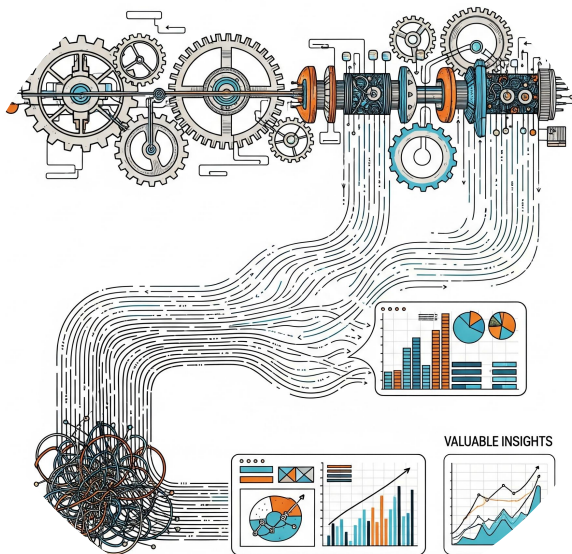
A data pipeline is a series of automated steps that move raw data from various sources to a destination, like a data warehouse, transforming it along the way to make it ready for analysis.

01 — Data pipelines are crucial for processing data

02 — Essential for modern data-driven systems

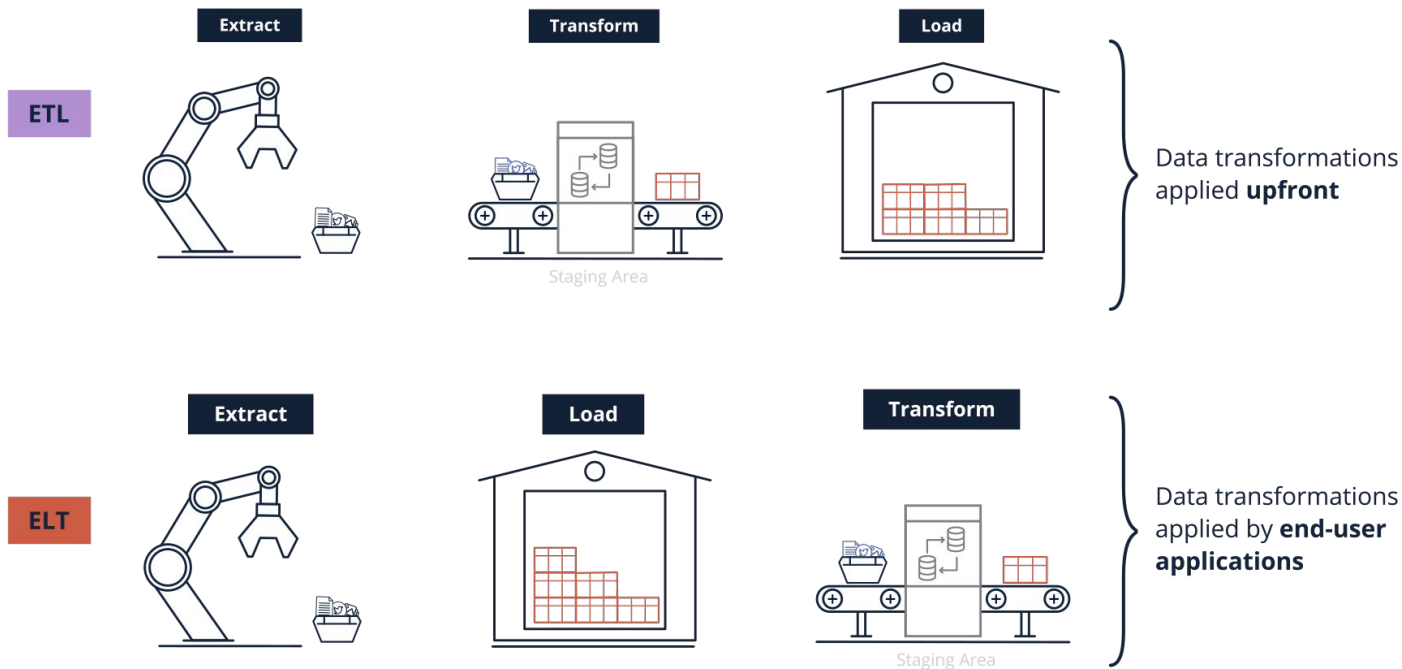
03 — They automate the flow of data

04 — Pipelines transform raw data into insights



Traditional ETL vs. Modern ELT Processes

ETL refers to a method of **extracting** and **transforming** data **from a source**, like a database, and **loading** the data into a new host source, like a data warehouse. Due to advances in technology, the modern **ELT** process came into existence, where the data **transformation** takes place **after loading** the data.

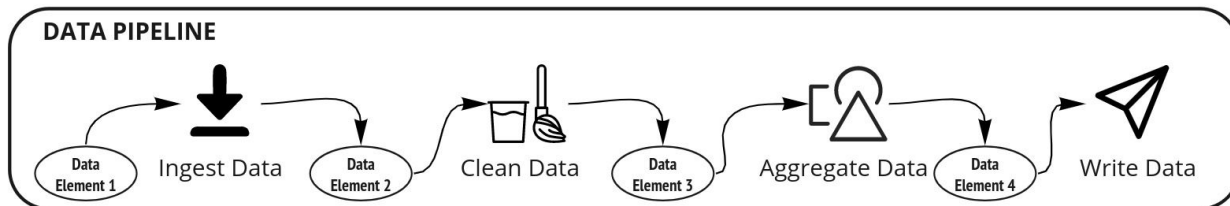
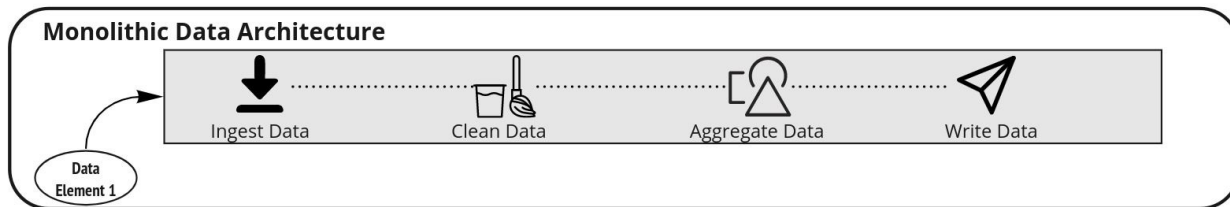


ETL Orchestration and data pipelining



Data pipelines provide greater throughput by processing more data elements per unit time.

Monolithic applications arise when **code and software components are combined** into a single application. Given a new data element, **the entire script** is executed. This script could contain the authorization, ingestion, cleaning, aggregation, writing, and integration of the data element.



A data pipeline is like **an assembly line** of individual data processing units. Each step in the pipeline functions as an individual worker, introducing a number of efficiencies over monolithic applications. Here, **four times as many data elements** can be processed than with a single monolithic application.

Data pipeline principles



Not all pipelines are created equal. Good data engineers ensure pipelines adhere to three core principles.

Adaptable

Data sets come in various shapes, sizes and formats. During the design phase of a pipeline, **we don't always have all of the non-functional requirements** for the system.

Data pipelines should be:

- designed for any data source and type;
- designed for changes in data volume, variety and velocity; and
- highly scalable.

Automated

Fully automated data pipeline:

- extract data from a source system;
- clean the data;
- perform quality control;
- transform the data in various ways;
- join the data to other data sources; and
- load the data to a destination on a regular schedule, or in response to an external event, **without human intervention**

Observable

Data pipelines should not fail. But when they do, we have to be able to find and isolate the failure(s) quickly. Tests and monitoring should allow us to **rapidly correct errors and patch our production systems**.

Robust and observable pipelines mean you have to:

- establish inherent data quality logging and testing;
- perform validation and testing; and
- Configure monitoring and alerting.

Data Pipeline Triggers

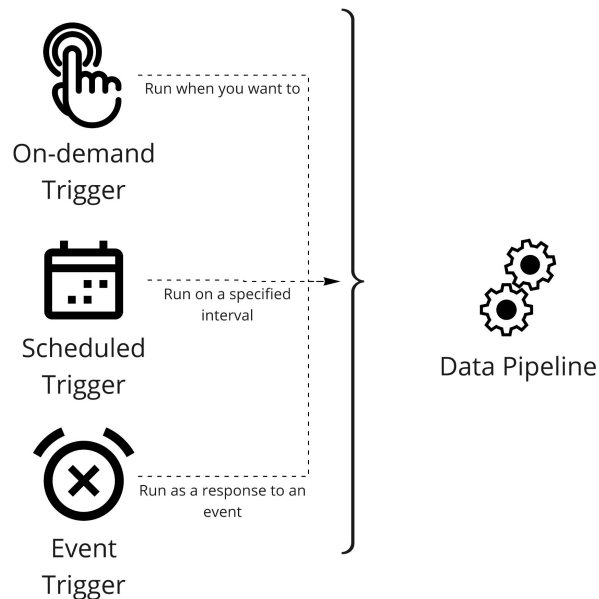
Pipeline triggers are required to initiate the running of a pipeline.

There are three different ways to activate a pipeline: **manual triggers**, also known as **on-demand** triggers; and **automated triggers**, which include **scheduled** and **event-based** triggers.

Run the pipeline **at will**, as and when you need to. It is especially **useful for testing** whether the pipeline is fully functional.

The **running** of the pipeline will be initiated according to a **specific schedule** or **routine**. This might be every 10 minutes, hourly, daily or even on specific dates.

An event could be a **file** being **created**, an **email** being **received**, or even an **alarm threshold** being **reached**. The **running** of the pipeline will be initiated by a **notification** that describes the change of a state within the system.



Data Pipeline Observability

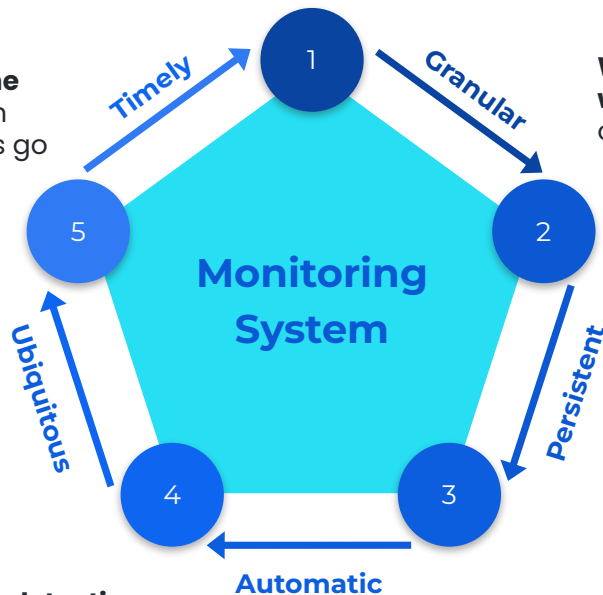


Observability refers to having a view of the **health** of a data pipeline or data solution by **measuring** the **internal metrics** of the solution. This includes **monitoring**, **tracking**, and **responding** to a data pipeline's **performance** to reduce and prevent downtime.

Monitoring should be performed in **real-time** displaying the 'as-is' state of the application and allowing immediate action when things go wrong.

We should be measuring the **performance** of code, **infrastructure**, and **runtime**. Failure in any of these three elements can lead to a degradation in performance.

The use of automated systems for **anomaly detection**, **problem identification**, etc., to reduce the time spent actively monitoring a system.



Where is the problem is occurring; and **with which piece of code** the problem is occurring.

The **lineage** should be **preserved** for all data pipeline tasks so that the exact time a problem arose can be identified.